

# توسعه مبتنی بر مشخصات در سطح تولید

نقشه راه گردش کارهای مقیاس پذیر و تحول تیم:  
از نمونه شهودی تا واقعیت تولید

بر پایه مقاله لی بونسترا

# «کدنویسی شهودی (Vibe coding)» به معنای «شهودی در تولید» نیست.

ما عامل‌ها (agent) را با تمام توانشان به کار می‌گیریم، اما همه‌چیز کاملاً قصدشده و کنترل‌شده است.

# توهم سرعت

یک ویرایشگر هوشمند می‌تواند به سرعت هزار خط کد خوب مستند شده بیرون بدهد. حس می‌کنید لشکری از کارآموز استخدام کرده‌اید که نه می‌خواهند و نه شکایت می‌کنند. اما یک نکته هست:

## توهم، هزار خط می‌سازد

وقتی عامل دچار توهم (hallucination) می‌شود، فقط یک اشکال نمی‌سازد؛ هزار خط منطقِ «هم‌حس اما کارکردشکسته» تولید می‌کند.

## نسبت اشکال به کد

هوش مصنوعی کد را بسیار سریع‌تر می‌نویسد – و می‌تواند خطاهای بالقوه را هم با سرعتی بی‌سابقه تولید کند.

**و نتیجه:** اگر بازبینان انسانی زیر سیل پول‌ریکوئست‌ها (pull request) غرق شوند، سرعت نوشتن بی‌ربط می‌شود. فرایند واقعاً سریع‌تر نشده؛ فقط انبوه بزرگ‌تری برای مرتب‌کردن بعدی ساخته‌اید.

# گلوگاه جابه‌جا شده، نه حذف

## پیش‌تر

گلوگاه: نوشتن کد



## امروز

گلوگاه: بازبینی، تست (test) و یکپارچه‌سازی

## سهم انسان

معماران که مشخصات (spec) تست، مشخصات یکپارچگی و نقشه‌های عملیاتی را می‌نویسند.

## سهم هوش مصنوعی

کار سنگین: نوشتن کد تست، انجام یکپارچه‌سازی، و مدیریت جزئیات ریز عملیاتی.

**و یک نکته امیدبخش:** چون پیاده‌سازی دیگر گلوگاه اصلی نیست، می‌توان هوش مصنوعی را به نوشتن پوشش (coverage) تستی گماشت که هیچ انسانی در همان بازه زمانی نمی‌توانست بنویسد.

# نقشه امروز – و نقشه کل دوره

## جلسه ۱

مهار (harness) و طیف

## جلسه ۲

ابزار (tool) و تعامل پذیری

## جلسه ۳

مهارت (skill) و حافظه رویه‌ای  
(Procedural memory)

## جلسه ۴

امنیت و ارزیابی (evaluation)

## جلسه ۵ – امروز

مشخصات، فرهنگ تیم، و شبکه ایمنی اعتماد صفر: همه چیز در یک گردش کار (workflow) واقعی.

**نکته مهم:** این تکنیک‌ها را باید از همان ابتدا به کار برد، نه وقتی نیمی از راه را رفته‌اید.

# توسعهٔ مبتنی بر مشخصات

در دنیای سنتی به ما آموختند «کد-اول» باشیم: ایده‌ای مبهم، ویرایشگری باز، و تایپ تا وقتی چیزی کار کند.

# نقش تازه توسعه‌دهنده: معمار فنی

بیشتر وقت امروز صرف نوشتن مشخصات باکیفیت می‌شود: دستوره‌ای فنی دقیقی که به عامل می‌گویند دقیقاً چه بسازد.

**و بخش حیاتی ماجرا:** کد حالا **دورریختنی** است. اگر مشخصاتی محکم بنویسید، کل کدبیس بارها و بارها بازتولیدشدنی است. حتی می‌توانید به عامل بگویید کل پروژه را در یک بعدازظهر از یک زبان به زبان دیگر ببرد.

**پیامد روان‌شناختی:** چون کد دورریختنی (Disposable code) است، آن دلبستگی عاطفی قدیمی وجود ندارد. وقتی دوازده ساعت صرف دیباگ (debug) یک نقطه‌ویرگول نکرده‌اید، از دورانداختن و شروع دوباره هم نمی‌ترسید.

# مشخصات خوب: ستاره شمالی معماری

مشخصات خوب از پاره‌پاره‌شدن زمینه (Context fragmentation) جلوگیری می‌کند – معادل دیجیتالی بازی «تلفن»، جایی که عامل رشته‌ماجر را گم می‌کند چون دارد به عکس‌های قدیمی فایل‌ها نگاه می‌کند.

## چه کسی می‌نویسدش

می‌توانید از هوش مصنوعی به‌عنوان هم‌نویسنده یا ویراستار و بازبین مشخصات استفاده کنید.

## کجا زندگی می‌کند

معمولاً درون خود کدبیس: فایل‌های مارک‌داون یا YAML در پوشه `specs/` . منبع حقیقت، هم برای انسان و هم برای عامل.

**نکته عملی:** طراحی فنی را جایی بنویسید که دیگران بتوانند بخوانند و بازبینی کنند. این حالا مهم‌تر از همیشه است: ترجیح می‌دهید **انسانی** نقص منطقی طرح را بگیرد، تا اینکه صبر کنید هوش مصنوعی هزاران خط کد شکسته تولید کند.

# آناتومی مشخصات تولیدی

۱

## طراحی فنی کامل

فقط نگویید «صفحه ورود بساز». تجزیه‌اش کنید: نیازمندی‌ها، اسکیمای پایگاه داده (ساختار داده‌هایتان) و مشخصات رابط‌ها (قراردادهایی که اجازه می‌دهند بخش‌های مختلف نرم‌افزار با هم حرف بزنند).

۲

## کمک بصری

نمودارها، و فهرست ابزارها و کتابخانه‌ها همراه با شماره نسخه.

۳

## اطلاعات زمینه‌ای

«چرا» را پشت «چه» بگویید. این کمک می‌کند عامل رو به جلو فکر کند و گام‌هایی را که احتمالاً لازم خواهید داشت پیش‌بینی کند.

۴

## سناریوها

خوب چه شکلی است، بد چه شکلی است، و حالت‌های مرزی کدام‌اند.

# کدام قالب؟ مالیات قالب واقعی است

پژوهش‌ها نشان می‌دهند عامل‌های مدل‌زبانی به نحوهٔ قالب‌بندی دستورها حساسیت شدیدی دارند – تا حد افت چشمگیر عملکرد با فایل‌های مارک‌داون عمومی و بهینه‌نشده.

**راہبرد پیشنهادی – ترکیبی:** از سرتیترهای تمیز مارک‌داون برای لنگرانداختن توجه مدل استفاده کنید، اما برای هر پیکربندی ساختارمند یا اسکیمای داده‌ای که **عمق تودرتویی بیش از سه** دارد، به YAML سوئیچ کنید.

| قالب | دقت تجزیه (decomposition) در ساختار عمیقاً تودرتو |
|------|---------------------------------------------------|
| YAML | بالا ترین                                         |
| JSON | متوسط                                             |
| XML  | پایین ترین                                        |

# توکن یک قید فیزیکی است

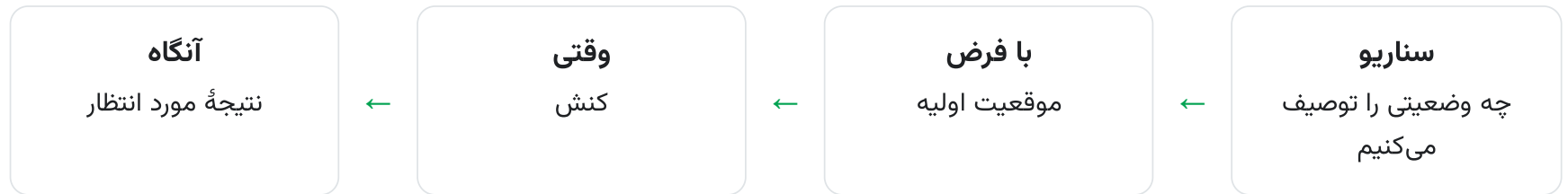
مدل‌های زبانی ساختار داده را تفسیر نمی‌کنند؛ متنِ نشانه‌گذاری‌شده را پردازش می‌کنند. هر نویسه‌ای که می‌فرستید به نشانه‌ها شکسته می‌شود و هر نشانه بودجه، زمان و ظرفیت زمینه (context) مصرف می‌کند.

**پیامد عملی:** هر فاصله اضافی در یک بلوک عمیقاً تودرتو، و هر دستور (instruction) تکراری، چرخه پردازش و ظرفیت توجه را در همان حلقه‌های (loop) چندنوبتی می‌بلعد که عامل در آن‌ها برنامه‌تان را می‌سازد، می‌آزماید و ویرایش می‌کند.

**پس:** با پوشه `specs/` نه مثل مستندات، بلکه مثل **یک مجموعه دستور کامپایل شده و کم‌چرب** رفتار کنید که مارک‌داون خوانا برای انسان را با بلوک‌های هدفمند و تخت YAML متوازن می‌کند.

# توسعه رفتارمحور: ابزار نهایی رفع ابهام

مشخصات رفتارمحور، ایده‌های مبهم انسانی را به طراحی معماری دقیقی بدل می‌کند که عامل می‌تواند بدون حدس‌زدن بسازدش. روشی که با زبان طبیعی ساده و ساختارمند توصیف می‌کند سامانه از دید کاربر دقیقاً چگونه باید رفتار کند – پیش از آنکه هیچ کدی نوشته شود.



**چرا کار می‌کند:** مدل را وادار می‌کند به صورت **موقعیت** ← **کنش** ← **نتیجه** فکر کند، و همین، کدنویسی شهودی را کاملاً حذف و عامل را روی ریل نگه می‌دارد.

## نمونه یک سناریوی رفتارمحور

Scenario: Retry a failed payment within the allowed window

Given a payment with status "failed" and 2 previous attempts

And the failure reason is "network\_timeout"

When the retry job runs within 24 hours of the original attempt

Then a new attempt is created with attempt\_number 3

And the original transaction id is preserved

And no duplicate charge is issued

Scenario: Refuse retry after the attempt limit

Given a payment with status "failed" and 3 previous attempts

When the retry job runs

Then no new attempt is created

And the payment is moved to status "requires\_manual\_review"

**توجه کنید:** این متن همزمان مشخصات است، هم پرونده تست، و هم دستور دقیق برای عامل. سه کار با یک نوشته.

## مشخصات رفتارمحور خودتان را بنویسید

۱

### طراحی فنی

نیازمندی‌ها، اسکیمای داده، قرارداد رابط‌ها.

۲

### نسخه‌ها

هر کتابخانه با شماره نسخه صریح.

۳

### چراها

زمینه تصمیم‌ها، نه فقط خود تصمیم‌ها.

۴

### سه سناریو

مسیر (trajectory) موفق، مسیر خطا، حالت مرزی (edge case).

**معیار پذیرش:** نفر کنار دستی باید **فقط با خواندن مشخصات** بتواند بگوید سامانه در هر سه سناریو دقیقاً چه می‌کند. هر جای ابهام، جایی است که عامل حدس خواهد زد.

# دستورها کجا زندگی می‌کنند؟

ریختن یک سند طراحی صد صفحه‌ای در پنجره گفت‌وگو، بودجه زمینه کوتاه‌مدت را به سرعت تمام می‌کند، تأخیر (latency) را بالا می‌برد و زمینه را پاره‌پاره می‌کند.

# چهار جایگاه دستور

| جایگاه                     | ماهیت                          | چه چیزی آنجا می‌رود                                                                                                                                                                                                    |
|----------------------------|--------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| رابط گفت‌وگو               | کوتاه‌عمر، ویژه نشست (session) | فقط هماهنگی سطح بالا و حلقه بازخورد فوری. مثلاً: «طراحی specs/payment_retry.md را بخوان و تست‌های واحد شکست‌خورده سناریوی ۳ را بساز.»                                                                                  |
| پوشه مشخصات                | ویژه کار، در کنترل نسخه        | طراحی فنی، سناریوهای رفتارمحور، قرارداد رابط‌ها و اسکیمای ساختارمند. عامل این دایرکتوری را پویا (Static) نمایه می‌کند و بی‌نیاز از چپاندن دستی درخواست، کد را می‌سازد و راستی‌آزمایی می‌کند.                           |
| مهارت‌های عامل             | قابل استفاده مجدد، رفتارمحور   | گردش کارهای تخصصی و محرک‌دار. عادت‌های مهندسی تکرارشونده را می‌آموزند – مثلاً نگهداری خودکار فهرست تغییرات وقتی تغییری در کد تشخیص داده می‌شود.                                                                        |
| پرامپت‌های (prompt) سامانه | سراسری، هویت‌محور              | جایی که هوش مصنوعی دی‌ان‌ای مهندسی شما را می‌آموزد. فایل‌های زمینه سلسله‌مراتبی خوانده و به هم چسبانده می‌شوند: از پروفایل سراسری کاربر، به پیکربندی مشترک چندابزاری، تا مشخصات ریشه پروژه که بالاترین اولویت را دارد. |

# پنج حالت اجرا، پنج ذهنیت

فقط یک راه برای تبدیل مشخصات به کد وجود ندارد.

# ۱. تولید پروژه – معمار

در این حالت، اسکلت پروژه از صفر ساخته می‌شود.

- **بدون حالت بی‌محابا (YOLO mode):** صریحاً به عامل بگویید فوری کد نزنند. اول باید ساختار پوشه‌ها و پشته فناوری را برای تأیید پیشنهاد بدهد.
- **تست، مستند و گزارش‌برداری را در درخواست بگنجانید** – گزارش‌برداری همان «جعبه سیاه» دیجیتالی است که ثبت می‌کند برنامه چه می‌کند.
- **برای هر کتابخانه شماره نسخه بدهید.** بدون آن، عامل ممکن است نسخه قدیمی‌تری پیشنهاد دهد، چون تاریخ پایان داده آموزشی‌اش در گذشته بوده است.

**مراقب باشید:** عامل‌ها به آسانی به داده آموزشی‌شان برمی‌گردند. شماره نسخه‌های پیشنهادی را همیشه دوباره واری کنید – یا مستندات به‌روز را در پوشه مشخصات بگذارید.

## ۲. تولید قابلیت – سازنده

پیاده‌سازی قابلیت روی کدبیس موجود.

### تأیید دستی هنگام ویرایش چندفایلی

وقتی چند فایل ویرایش می‌شود، تغییرها باید دستی تأیید شوند. دیدن تفاوت‌ها – فهرستی که دقیقاً می‌گوید چه خط‌هایی اضافه یا حذف می‌شود – درون ویرایشگر اهمیت دارد.

### تطبیق با سبک موجود

از عامل بخواهید سبک موجود را رعایت کند: الگوهای نام‌گذاری، و اینکه کد چطور خط‌ها را مدیریت می‌کند.

### ۳. رفع اشکال – متخصص جرم‌شناسی

وقتی چیزی خراب شد، حالت جرم‌شناسی لازم است. هدف، تحلیل علت ریشه‌ای و ترمیم جراحی است.

#### پرامپت علامت‌محور

«دکمه کار نمی‌کند.»

#### پرامپت شاهد‌محور

«گزارش‌هایی که با این فرمان گرفتم، خطای ۴۰۳ نشان می‌دهند.»  
سپس جریان را توضیح بدهید: «درخواست به متعادل‌کننده بار  
می‌رسد ← لایهٔ احراز هویت (identity) سرایند را حذف می‌کند ←  
غلاف شکست می‌خورد.»

- همیشه اول یک تست واحد شکست‌خورده یا فرمان بازتولید بخواهید، و آن پرونده را در کدبیس نگه دارید تا اشکال برنگردد.
- قید (constraint) سخت بگذارید که عامل فقط علت ریشه‌ای را درست کند؛ کد بی‌ربط نباید «تمیزکاری» شود، چون بازبینی را پیچیده می‌کند.
- تغییر نام متغیرها پذیرفتنی است، اما باید کاری جداگانه باشد، نه بخشی از رفع اشکال.

# مرورگر داخلی: تست سرتاسری زنده

برخی محیط‌های عامل محور مرورگری داخلی دارند که به عامل اجازه می‌دهد خودگردان (autonomous) برنامه محلی را اجرا کند، با رابط زنده تعامل کند و اصلاح‌های بصری را در زمان واقعی راستی‌آزمایی کند.

**و نکته امنیتی مهم:** این زیرعامل (subagent) درون نمایه مرورگری کاملاً ایزوله و سندباکس‌مانند اجرا می‌شود که ورودهای فعال یا نشست‌های شخصی شما را نمی‌بیند – عملاً یک محیط ناشناس تمیز. همین از نشت (leak) زمینه و عملیات تصادفی روی حساب واقعی جلوگیری می‌کند.

**پیامد:** درخواست‌های دیباگ می‌توانند بسیار فراتر از تحلیل ایستای کد بروند: به مرورگر داخلی دستور گام‌به‌گام بدهید تا دقیقاً نشان دهد چه چیزی از نظر بصری خراب است و چه چیزی روی صفحه باید درست شود.

## ۴ و ۵ – نویسنده و کتابدار

### ۴. نوشتن مستندات – نویسنده

در توسعه مبتنی بر مشخصات، مستندات منبع حقیقت است. اگر مستندات و کد هم‌گام نباشند، هوش مصنوعی شروع به توهم می‌کند.

در مهارت‌های عامل مشخص کنید که دستورهای فایل راهنما و فهرست تغییرات همیشه باید نگهداری شوند. از قالب‌های استاندارد مستندسازی درون‌کد استفاده کنید تا هم انسان و هم هوش مصنوعی بدون خواندن هر خط منطق بفهمند یک تابع چه می‌کند.

### ۵. مهندسی داده – کتابدار

هنگام پرس‌وجوی جدول‌ها یا جابه‌جایی فایل‌ها، نقش شما مهندس داده است. از افزونه‌های ویرایشگر برای دسترسی مستقیم به داده ابری استفاده کنید.

و یک قاعده ساده اما مهم: از عامل بخواهید همیشه پرس‌وجو یا فرمان دقیقی را که برای تولید خروجی به کار برده نشان بدهد.

# فرهنگ تیم و تحول فرایند

چسباندن این فناوری به گردش کاری بیست ساله و انتظار پرواز، مثل نصب موتور جت روی درشکه است.

## سه دسته شکست فرایندی

### پاره پاره شدن زمینه

در غیاب یک توسعه دهنده، هم تیمی متغیری را در فایل مشترک تغییر نام می دهد؛ عامل که به عکس قدیمی استناد می کند، کدی می سازد که تابعی دیگر موجود را صدا می زند.

### گره خوردگی بازبینی

پول ری کوئستی غول آسا به عروسک روسی زیر درخواست ها بدل می شود: اولی بدون دومی ادغام نمی شود، دومی به سومی نیاز دارد، و سومی پشت بازبینی در منطقه زمانی دیگری گیر کرده است.

### تعارض های ادغام

چند توسعه دهنده ظرف یک ساعت روی یک فایل فرود می آیند.

# پنج راهبرد یکپارچگی پرسرعت

- **خلاصه و ارزیابی مخاطره بسته‌بندی‌شده:** هر پول‌ریکوئست باید نمای تولیدشده‌ای از آنچه تغییر کرده، نقاط محتمل شکست و ارزیابی مخاطره داشته باشد.  
تا بازیبن انسانی به‌جای گم‌شدن میان خط‌ها، روی اثر معماری تمرکز کند.
- **بازتعریف مالکیت:** تمرکز بازیبنی انسانی را از ایرادگیری سبکی روی کد دورریختنی، به تضمین یکپارچگی نقشه‌های معماری منتقل کنید.  
دغدغه‌های سبکی با ابزارهای خودکار و کتاب‌سبک‌های مشترک حل می‌شوند.
- **تأیید مشروط (Conditional LGTM):** بازیبن، پول‌ریکوئست را مشروط به گذراندن همهٔ تست‌های خودکار تأیید می‌کند؛ اگر تست‌ها سبز شدند، کد خودکار ادغام می‌شود.  
این تأخیرهای دوازده‌ساعته میان منطقه‌های زمانی را حذف می‌کند.
- **فرهنگ بدون سرزنش:** در محیط پرسرعت، کسی که بیشترین کد را تولید می‌کند اغلب قربانی سرزنش اشکال‌ها و تعارض‌ها می‌شود. این مسائل را به **فرایند یکپارچگی شکسته** نسبت بدهید، نه به فرد.
- **بازیبنی کد به‌دست‌عامل:** مهارتی بسازید که بازیبنی را برایتان انجام دهد – و حتی مهارتی که به بازیبنی‌ها پاسخ بدهد.

## نمونه مهارت بازبینی کد

Act as a Senior Software Engineer and Security Researcher.

Review the provided PR or diff using these strict criteria:

1. Critical Vulnerabilities: hardcoded secrets, SQL injection, XSS, or broken authentication.
2. Logic & Efficiency: off-by-one errors, infinite loops, redundant API calls.
3. Readability: better naming, break down mega-functions.
4. Edge Cases: What if the input is null? What if the network fails?

Output Format:

- Description: what is this PR doing? Explain in detail.

ISSUES:

- Critical: (stop-ship issues)
- Warnings: (code smells or style issues)
- Best Practices: (specific lines to refactor)
- Quick Win: (one-sentence biggest improvement)

When there are no issues, return the description and "LGTM".

# و چه کسی این را روی هر درخواست اجرا می‌کند؟

مهارت بازبینی وقتی اجرا می‌شود که شما از درون ویرایشگر صدایش بزنید. گام بعد، دیپلوی (deploy) آن به‌عنوان عامل تحلیل پیوسته است: سرویسی که ریپو (repository) را می‌پاید، به رویدادها واکنش نشان می‌دهد و یافته‌ها را بدون اینکه کسی بخواهد پست می‌کند.

**چه چیزی را می‌گیرد:** همان چیزهایی که انسان خسته عصر جمعه از دست می‌دهد — وابستگی‌ای با آسیب‌پذیری تازه‌کشف‌شده، یا یادداشتی از شش ماه پیش که بی‌سروصدا به شکاف امنیتی بدل شده است.

**و وقتی تیم شما با حجم بالا منتشر می‌کند،** بازبین پیوسته تنها چیزی است که همپای خروجی مقیاس می‌گیرد.

## سه لایهٔ بازبین خودکار

| لایه           | چیست                                                                                                     | موازنه                                                                                                                                                                                                                                 |
|----------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ۱ – مدیریت شده | سرویس آماده روی میزبان ریپو. فعالش می‌کنید و هر پولریکوئست بازبین می‌گیرد.                               | هیچ درخواستی نمی‌نویسید و هیچ زیرساختی مدیریت نمی‌کنید. اما نظرهای بازبینی <b>فروشنده</b> را می‌گیرید، نه نظرهای خودتان – و بازبین عمومی چیزی را که برای تیم با سبک خانگی سخت‌گیرانه یا نمای مخاطرهٔ حوزه‌ای اهمیت دارد از دست می‌دهد. |
| ۲ – ترکیبی     | کنشی در سی‌آی (CI) که خط فرمان عامل را در حالت غیرتعاملی اجرا می‌کند و نتیجه را به‌عنوان نظر پست می‌کند. | <b>مسیر میانی و نقطهٔ شروع درست برای بیشتر تیم‌ها.</b> رانتایم (runtime) مال ارائه‌دهندهٔ یکپارچه‌سازی شماس؛ اما درخواست‌ها، انتخاب مدل، سندباکس (sandbox) و معیارهای بازبینی مال شماس است.                                            |
| ۳ – سفارشی     | عاملی که خودتان می‌سازید و مستقر می‌کنید، با نشست و حافظهٔ ماندگار و اتصال به رویدادهای ریپو.            | وقتی درست است که بازبین باید زمینه را در یک بازسازی چنددرخواستی نگه دارد، کار بزرگ را به ده‌ها زیرکار بشکند، یا با عامل‌های دیگر هماهنگ شود. اما ارزیابی، رصدپذیری، هزینه و نوبت‌کاری هنگام خرابی هم مال شماس است.                     |

## سه پرسش که لایه را تعیین می‌کنند

۱

معیار بازبینی‌تان چقدر ویژه است؟

عمومی ← لایه ۱.

ویژه تیم یا رپیو ← لایه ۲ یا ۳.

۲

آیا عامل باید چیزی را میان اجراها  
به یاد بسپارد؟

نه ← لایه ۱ یا ۲.

بله (حافظه کدبیس، زمینه چنددرخواستی)

← لایه ۳.

۳

بدترین حالت چیست؟

نظری پرحرف ← هر لایه‌ای.

پس‌رفتی که ادغام شود یا رازی که نشت

کند ← لایه ۳، با سرور سیاست (Policy

Server) پیش از هر فراخوان ابزار (tool

call).

**قاعده انتخاب: پایین‌ترین لایه‌ای را بردارید که آنچه اهمیت دارد را پوشش می‌دهد.** بیشتر تیم‌ها لایه خود را همان لحظه‌ای کشف می‌کنند که بازبین مدیریت‌شده چیز خاصی را از دست می‌دهد.

## لایه ۳ در مقیاس کامل: درک گرافی کد

روی کدبیزی با ده‌ها میلیون خط، بارکردن کد به شکل متن تخت در کانتکست ویندو (context window) جا کم می‌آورد، و بازیابی (retrieval) استاندارد ساختاری را حذف می‌کند که کد را خوانا می‌سازد: کلاس به فایلی تعلق دارد، و فراخوان تابعی به سندی که ده سال پیش نوشته شده ردیابی می‌شود.

### پیمایش گراف

برای پرسش‌های ساختاری: «هر تابعی که به‌طور متعددی فلان تابع را صدا می‌زند.»

### جست‌وجوی بُرداری

برای پرسش‌های معنایی: «کدی پیدا کن که کاری را می‌کند که این پاراگراف توصیف می‌کند.»

### جست‌وجوی تمام‌متن

برای تطبیق دقیق شناسه‌ها.

**ترکیب این سه** به پرسش «اگر این را عوض کنم چه چیزی خراب می‌شود؟» با نقشه اثر دقیق پاسخ می‌دهد، نه با حدسی مطمئن.

## نیمه دوم: تجزیه

یک عامل که به او بگویید «این ماژول را بازسازی کن» شکست می‌خورد. همان کار را روی یک خط لوله زیرعاملی تقسیم کنید و کار مدیریت‌پذیر می‌شود:



**نتیجه گزارش‌شده:** در کدبیس‌های میلیون‌خطی، کار بازسازی معادل از دو هفته به چند ساعت رسیده است. این لایه ۳ در مقیاس کامل است: نه عاملی که درخواست‌ها را می‌پاید، بلکه عاملی که **سامانه‌ای را می‌فهمد که آن درخواست‌ها در آن زندگی می‌کنند.**

# پایداری: خستگی تأیید

## پدیده تازه

پژوهش‌ها نشان می‌دهند کاربران پرکار هوش مصنوعی به مراتب بیشتر در معرض فرسودگی شدیدند. زیر جریان مداوم تأییدهای خرد – بهبود یک خط، تنظیم یک فراخوان ابزار – توسعه‌دهنده بازتابی روی «تأیید» کلیک می‌کند.

## از نظارت مداوم به مرزهای ساختارمند

ساعات‌های آرام دیجیتال: مرز بگذارید تا درخواست‌های تأیید به عصرها و آخر هفته‌ها نشت نکنند.

جلسه‌های بینش عامل: هفتگی، توسعه‌دهندگان الگوهایی را که همتای هوشمندشان کشف کرده به اشتراک می‌گذارند و کشف‌های منفرد به دانش سازمانی مشترک بدل می‌شود.

**خطر واقعی:** شکلی از خستگی کم‌شدت که در آن تیم فقط برای همپا ماندن با سرعت ماشین، از واریسی کارش دست می‌کشد.

# توسعهٔ اعتماد صفر: ساختن شبکهٔ ایمنی

و حالا درسی دربارهٔ آنچه وقتی عامل بدون گاردریل (guardrail) کافی عمل می‌کند اتفاق می‌افتد.

# داستان پنجاه ایمیل



**مخاطره توهم زمینه (Context hallucination):** وقتی هوش مصنوعی داده کافی ندارد، گاهی شکاف‌ها را با هر رشته‌ای که در زمینه‌اش هست پر می‌کند – از جمله اطلاعات حساسی مثل نشانی‌های ایمیل سخت‌گذشته یا نشانی‌های خصوصی.

عامل داشت دستورش را با داده‌ای که در اختیار داشت انجام می‌داد، بدون هیچ واریسی‌ای بر اینکه آیا باید این کار را بکند یا نه.

این هستهٔ مخاطره در سامانه‌های خودگردان است. گارد ریل‌ها اختیاری نیستند؛ همان چیزی‌اند که نمی‌گذارند ابزاری مفید به ابزاری غیرقابل‌پیش‌بینی بدل شود.

# پارادوکس گاردریل

عامل‌ها باید آن قدر خودگردان باشند که مسائل پیچیده را حل کنند، اما نمی‌توان ریسک یاغی‌شدنشان را در محیط سازمانی پذیرفت.

**مثال:** عاملی که مأمور «حل اختلاف‌های مشتری» است. برای مؤثر بودن، به داده مشتری، ابزار ایمیل و سامانه‌های داخلی نیاز دارد. اما چالش این است که مطمئن شویم تصادفی کل پایگاه داده را ایمیل نمی‌کند یا کد اختصاصی را به اشتراک نمی‌گذارد.

**و چرا سخت‌کد کردن قید در پرامپت سامانه جواب نمی‌دهد:** عامل‌های خودگردان را مدل‌هایی می‌رانند که **احتمالاتی‌اند، نه قطعی (deterministic)**. زمینه‌ها سرریز می‌کنند و می‌شود عامل را با تزریق پرامپت «قانع» کرد که قواعد را دور بزند. برای سکوی سطح تولید، به حکمرانی **بیرونی و دست‌نیافتنی** نیاز دارید.

# پنج لایه شبکه ایمنی

۱

## سندباکس

اجرای کارهای عامل محور درون کانتینرهای زودگذر (ephemeral) و کم‌امتیاز، ایزوله از شبکه اصلی و فایل سیستم حساس. اگر عامل فریب بخورد، آسیب به نمونه‌ای دورریختنی محدود می‌ماند.

۲

## انسان در حلقه

دروازه‌های (gate) واری برای کنش‌هایی با نمای مخاطره مشخص: انتشار در تولید، تغییر اسکیمای پایگاه داده، آغاز تراکنش مالی.

۳

## پوشش تست تولیدشده

ماشین را مأمور نوشتن همان تست‌هایی کنید که خروجی خودش را اعتبارسنجی می‌کنند. اول تست شکست‌خورده، بعد اصلاح.

۴

## ارزیابی

تست واحد می‌پرسد «آیا تابع مقدار درست را برگرداند؟» – پاسخی صفر و یکی. ارزیابی می‌پرسد «آیا رفتار عامل دست‌کم به‌خوبی مبناست؟» تست‌ها پس‌رفت (regression) قطعی را می‌گیرند، اما ارزیابی انحراف (drift) رفتاری را می‌گیرد.

۵

## سرور سیاست

حکمرانی بیرونی که کنش‌ها را پیش از رسیدن به سامانه‌های بیرونی رهگیری (interception) می‌کند.

# چرا ارزیابی، و نه فقط تست؟

تست‌های سنتی برای سامانه‌هایی که خروجی‌شان تولید می‌شود و نه محاسبه، کافی نیستند.

**مسئله:** عامل – یا هر جزء مدل‌محور دیگری – می‌تواند صد تست واحد را روی ابزارهایش بگذراند و همچنان به شکل تماشایی شکست بخورد: ابزار اشتباه را انتخاب کند، پاسخی حیاتی را بدجور بازنویسی کند، یا واقعیتی را توهم کند.

**راه‌حل:** جایگزین کردن ادعاهای صفر و یکی با **قضاوت‌های نمره‌دار و باندهای تحمل**: نمره‌ای از داور (judge) مدل‌زبانی، واریسی مسیری که تنوع ترتیب فراخوان ابزار را تحمل می‌کند، و دروازه‌ای که وقتی کیفیت زیر حاشیه‌ای قابل‌پیکربندی بیفتد فعال می‌شود – نه وقتی ادعایی نقض شود.

**ویک بازتعریف مهم:** حاشیه خطا نقصی نیست که باید حذف شود؛ ویژگی ذاتی مدل است و راهبرد تست باید با آن کنار بیاید.

# سرور سیاست: دو لایه

## دروازه معنایی – داور هوشمند

مدلی تخصصی که قصد (intent) و محتوای کنش پیشنهادی را در برابر راهنماهای حریم خصوصی به زبان طبیعی بازرسی می‌کند. برای وقتی که ابزار مجاز است اما نحوه استفاده سیاست (policy) را نقض می‌کند: مدیر می‌تواند ایمیل بفرستد، اما نباید اطلاعات هویتی پوشانده نشده بفرستد.

## دروازه ساختاری – چراغ راهنما

قواعد قطعی بر پایه نقش و محیط. وارسی‌هایی سریع و صفر و یکی – مثلاً نقش «بیننده» نمی‌تواند از ابزار ارسال ایمیل استفاده کند. این لایه بدون پرسیدن از هیچ مدلی، از نقض معماری جلوگیری می‌کند.

**چرا هر دو لازم‌اند:** نمی‌توانید هر نشت ممکن اطلاعات هویتی را با عبارت باقاعده بگیرید. اینجا دقیقاً جایی است که قواعد ساختاری شکست می‌خورند.

# فایل سیاست و جریان اجرا

```
environments:
  localhost:
    blocked_tools:
      - send_email

roles:
  viewer:
    allowed_tools:
      - list_files
      - read_file
```

## ۱. واریسی ساختاری

آیا این ابزار برای این نقش و محیط مجاز است؟



## ۲. واریسی معنایی

آیا آرگومان‌ها امن‌اند؟



## ۳. اجرا

اگر هر دو گذشتند، ابزار اجرا می‌شود؛ وگرنه پیام «نقض سیاست» به عامل برمی‌گردد تا خودش را اصلاح کند یا با ظرافت شکست بخورد

## بهداشت زمینه و پاک‌سازی درخواست

برای مهار مخاطرهٔ توهّم زمینه، میان‌افزاری بگذارید که اطلاعات هویتی را می‌پوشاند و جانشین تزریق می‌کند. با جایگزین کردن اطلاعات شناسایی‌پذیر با جانشین‌های عمومی در قالب‌ها، مطمئن می‌شوید عامل روی دادهٔ استریل‌شده کار می‌کند.

```
def resolve_context(template_str, override_state=None):
    """Replace [[VARIABLE_NAME]] with runtime state or env values."""
    state = override_state or {}

    def replacement(match):
        var = match.group(1).strip()
        if state.get(var) is not None:      # 1. runtime override
            return str(state[var])
        if os.environ.get(var) is not None: # 2. validated env var
            return os.environ[var]
        return match.group(0)              # 3. leave unresolved, no silent failure

    return re.sub(r'\[\[([^\]]+)\]\]', replacement, template_str)
```

# اعمال سراسری بهداشت زمینه

این ابزار باید مستقیم در خط لوله اجرای عامل به عنوان گام اعتبارسنجی سیم‌کشی شود: فراخوان‌های ورودی ابزار را پیش از اجرا رهگیری کنید تا همه رشته‌های تزریق‌شده پویا استریل شوند.

**نتیجه:** هر تلاش عامل برای اجرای کنشی (مثل فرستادن ایمیل یا پرس‌وجوی یک منبع ابری) رهگیری می‌شود. موتور، جانشین‌های عمومی را امن و بی‌صدا به دارایی‌های تستی مجاز ترجمه می‌کند و نیاز به سخت‌کد کردن اطلاعات حساس در مجموعه تست‌ها یا پرامپت‌های سامانه را کاملاً حذف می‌کند.

**اصل معماری:** این کار منطق اجرا را از منطق حکمرانی جدا می‌کند – تفکیک مسئولیتی حیاتی برای نرم‌افزار سازمانی.

# سندباکس در عمل

فراتر از پاک‌سازی (sanitization) رشته‌ها، امنیت واقعی محیط اجرای محدود می‌خواهد. حتی با پالایش دقیق خروجی، مدل ممکن است کدی تولید کند که از نظر نحوی معتبر اما از نظر منطقی مخرب است.

## گزینه تیمی و قابل حمل

فضای کاری عامل را در کانتینر (container) بگذارید: فایل کانتینر سفارشی بنویسید، اعتبارنامه ابری محدود تزریق کنید، و ابزار را وادار کنید کاملاً درون کانتینر اجرا شود.

## گزینه ساده

در بسیاری از محیط‌های عامل‌محور، سندباکس ترمینال با یک کلید در تنظیمات فعال می‌شود.

**در هر دو مدل:** اگر عامل فریب بخورد و اجرای مخربی انجام دهد، به خطای مجوز سختی در سطح هسته برمی‌خورد و سامانه میزبان کاملاً دست‌نخورده می‌ماند.

# پنج جلسه در یک تصویر

آنچه در این پنج هفته ساختیم، یک ابزار نیست؛ یک سامانه است.

# مسیری که آمديم

| جلسه | پرسش محوری                           | پاسخی که ساختيم                                                                              |
|------|--------------------------------------|----------------------------------------------------------------------------------------------|
| ۱    | چه چیزی واقعاً عوض شده؟              | گذار از نحو به قصد؛ طیف شهودی تا عامل محور؛ و اینکه عامل = مدل + مهار.                       |
| ۲    | عامل چطور با جهان حرف می زند؟        | پروتکل های باز به جای پوشش های سفارشی؛ ابزار، همکاری، رابط کاربری و تجارت.                   |
| ۳    | عامل چطور می داند «چگونه»؟           | مهارت به عنوان حافظه رویه ای، با افشای تدریجی (Progressive disclosure) و ارزیابی سخت گیرانه. |
| ۴    | چه چیزی خراب می شود و چطور می سنجيم؟ | هفت ستون امنیت، و هفت بُعد ارزیابی در دو محور مستقل اعتماد.                                  |
| ۵    | چطور این ها را به تولید می رسانی؟    | مشخصات به عنوان منبع حقیقت، فرایند تیمی بازطراحی شده، و شبکه ایمنی اعتماد صفر.               |

## جمع‌بندی

در کمتر از یک سال، چرخه توسعه نرم‌افزار به شکل چشمگیری سریع‌تر شده است. تماشای عاملی که تا ظهر هزار خط کد خوب مستند شده تولید می‌کند، پیشرفت مهمی است.

**اما آن سرعت، جابه‌جایی مهمی را آشکار کرد.** هوش مصنوعی گلوگاه تولید کد را حذف کرده و قید را به پایین‌دست برده: به انسان‌هایی که باید این خروجی را بازبینی، تست و یکپارچه کنند.

**و پاسخ در پرامپت‌های بهتر یا مدل‌های سریع‌تر نیست.** موفقیت به تحول پویایی تیم، پالایش همکاری با عامل‌ها، و تعیین مرزهای سخت‌گیرانه برای ابزارهایی بستگی دارد که هرگز نمی‌خواهند. چالش از تولید کد، به **هماهنگ کردن سامانه‌هایی** منتقل شده که کار را راستی‌آزمایی، یکپارچه و مستقر می‌کنند.

# پروژه پایانی دوره

یک قابلیت کوچک اما واقعی را از مشخصات تا انتشار ببرید و هر پنج جلسه را در آن نشان دهید. گزارش نهایی، حداکثر چهار صفحه.

1. **مهار:** فایل قواعد پروژه و جای این کار روی طیف.
2. **ابزار:** عامل به چه ابزارهایی وصل شد، با چه دامنه‌ای، و چرا این‌ها و نه بیشتر.
3. **مهارت:** دست‌کم یک مهارت با پرونده‌های ارزیابی‌اش.
4. **امنیت و ارزیابی:** یافته‌های بازبینی امنیتی و سنجه‌های (rubric) کیفیت.
5. **مشخصات و فرایند:** مشخصات، مهارت بازبینی، و شبکه ایمنی.

**معیار اصلی:** گزارش باید نشان دهد **تصمیم‌های آگاهانه** گرفته‌اید، نه اینکه همه توصیه‌ها را اجرا کرده‌اید. جایی که چیزی را **عمداً انجام نداده‌اید** و دلیلش را نوشته‌اید، به اندازه جایی که انجام داده‌اید ارزش دارد.

# تولید حل شده است. راستی آزمایی، قضاوت و جهت دهی، پیشه تازه اند.

سپاس از همراهی تان در این پنج جلسه.

پرسش ها، بازخوردها و پروژه های پایانی – همین حالا.